Starting a thread can fail by throwing an exception, so you need to ensure that any threads you've already started are stopped and cleaned up nicely in this case. This is achieved with a `try-catch` block that sets the `done` flag when an exception is thrown ❿, alongside an instance of the `join_threads` class from chapter 8 ❸ to join all the threads. This also works with the destructor: you can just set the `done` flag ⓫, and the `join_threads` instance will ensure that all the threads have completed before the pool is destroyed. Note that the order of declaration of the members is important: both the `done` flag and the `worker_queue` must be declared before the `threads` vector, which must in turn be declared before the `joiner`. This ensures that the members are destroyed in the right order; you can't destroy the queue safely until all the threads have stopped, for example.

The `worker_thread` function itself is quite simple: it sits in a loop waiting until the `done` flag is set ❹, pulling tasks off the queue ❺ and executing them ❻ in the meantime. If there are no tasks on the queue, the function calls `std::this_thread::yield()` to take a small break ❼ and give another thread a chance to put some work on the queue before it tries to take some off again the next time around.

For many purposes such a simple thread pool will suffice, especially if the tasks are entirely independent and don't return any values or perform any blocking operations. But there are also many circumstances where such a simple thread pool may not adequately address your needs and yet others where it can cause problems such as deadlock. Also, in the simple cases you may well be better served using `std::async` as in many of the examples in chapter 8. Throughout this chapter, we'll look at more complex thread pool implementations that have additional features either to address user needs or reduce the potential for problems. First up: waiting for the tasks we've submitted.

9.1.2. Waiting for tasks submitted to a thread pool